

Speciális hálózati folyam problémák

Modellezés és visszavezetések

Szerző: Nikházy László
Előadó: Varga Péter

2026. május 14.

Horváth Gyula diasorának felhasználásával

Emlékeztető: A standard hálózati folyam probléma

Mielőtt belevágunk a speciális esetekbe, röviden foglaljuk össze, miből indulunk ki!

Adott egy $G = (V, E, c)$ irányított, súlyozott gráf, egy s (forrás) és egy t (nyelő) pont. Olyan $f : E \rightarrow \mathbb{N}$ folyamfüggvényt keresünk, ahol a folyam értéke maximális, és:

1. **Kapacitás korlát:** Minden élen a folyam érvényes:
 $0 \leq f(e) \leq c(e) \quad (\forall e \in E).$
2. **Megmaradás:** Bármely $v \neq s, t$ pontra igaz, hogy amennyi folyam befolyik, annyi folyik ki:

$$\sum_{(u,v) \in E} f(u,v) = \sum_{(v,w) \in E} f(v,w)$$

A cél a következő problémák esetén: már ismert Ford-Fulkerson / Edmonds-Karp algoritmusunkat úgy alkalmazzuk, hogy a bemeneti gráfot átalakítjuk (kiegészítjük/módosítjuk).

1. Pontok kapacitás korláttal – A probléma

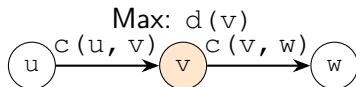
Eddig csak az éleknek volt kapacitása. Mi van, ha a csúcsokon is csak véges mennyiségű folyam haladhat át?

Bemenet: $G(V, E, c, d)$ súlyozott gráf:

- $c : E \rightarrow \mathbb{N}$ élkapacitások
- $d : V \rightarrow \mathbb{N}$ **pontkapacitások**

Olyan maximális f folyamot keresünk, amelyre a korábbi feltételeken felül a csúcsokra is teljesül:

$$\sum_{(u,v) \in E} f(u,v) \leq d(v) \quad (\text{minden } v \neq s, t\text{-re})$$

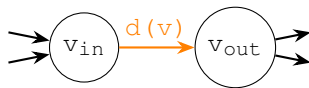
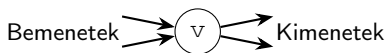


1. Pontok kapacitás korláttal – Megoldás

Trükk: "Hasítsuk ketté" a csúcsot! Minden v csúcsból csináljunk egy v_{in} és egy v_{out} csúcsot, és a pontkapacitást tegyük egy a kettő közötti irányított élre!

A visszavezetés lépései $\bar{G}$ gráfra:

- Minden $v \in V \rightarrow$ két pont:
 (v_{in}) és (v_{out}) .
- Belső él: $(v_{in}) \rightarrow (v_{out})$
 $d(v)$ kapacitással.
- Eredeti (u, v) él: Ebből
 $(u_{out}) \rightarrow (v_{in})$ él lesz, az
eredeti $c(u, v)$ kapacitással.



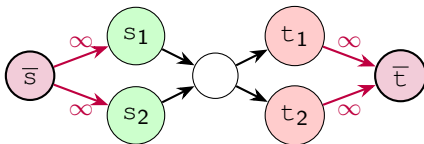
2. Több forrás, több nyelő

Mi történik, ha egy gyárhálózatban több raktárból (S csúcshalmaz) akarunk eljuttatni termékeket több boltba (T csúcshalmaz)?

Megoldás: Hozzunk létre egy **szuper-forrást** ($\bar{s}$) és egy **szuper-nyelőt** ($\bar{t}$)!

- $\bar{s}$ -ből húzzunk élet minden $s \in S$ forrásba ∞ kapacitással.
- Minden $t \in T$ nyelőből húzzunk élet $\bar{t}$ -be ∞ kapacitással.

Lefuttatva a maximális folyam keresést az $\bar{s} \rightarrow \bar{t}$ hálózaton, a probléma megoldva!



3. Hálózati folyam követelményekkel (Demands)

A csúcsoknak most már nem kapacitása van, hanem bizonyos mennyiségű árut **kínálnak** vagy **igényelnek**. Nincs globális s és t .

Bemenet: $G = (V, E, c, d)$, ahol

- $c : E \rightarrow \mathbb{N}$ az él kapacitása.
- $d : V \rightarrow \mathbb{Z}$ a csúcs követelménye.
- Ha $d(v) < 0$: a csúcs termel (kínálata van).
- Ha $d(v) > 0$: a csúcs fogyaszt (igénye van).
- Alapfeltétel: A teljes kínálat és kereslet egyensúlyban van:

$$\sum_{d(v) > 0} d(v) = \sum_{d(v) < 0} -d(v) = D.$$

Kérdés: Létezik-e olyan $f : E \rightarrow \mathbb{N}$ megengedett folyam, ami minden igényt kielégít? Azaz minden csúcsra a befolyó összfolyam és a kifolyó összfolyam különbsége megegyezik a csúcs követelményével:

$$\sum_{(u,v) \in E} f(u,v) - \sum_{(v,w) \in E} f(v,w) = d(v)$$

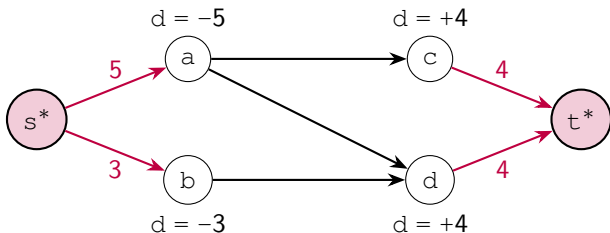
és $0 \leq f(e) \leq c(e) \quad \forall e \in E$ élre továbbra is

3. Megoldás: Szupercsúcsok a kínálatnak és keresletnek

Visszavezetés maximális folyamra: Vegyünk fel egy s^* és egy t^* pontot!

- Az összes termelőt rákötjük a szuperforrásra: (s^*, v) ha $d(v) < 0$, kapacitás: $-d(v)$.
- Az összes fogyasztót rákötjük a szupernyelőre: (v, t^*) ha $d(v) > 0$, kapacitás: $d(v)$.

Állítás: Akkor és csak akkor van megfelelő folyam az eredeti gráfban, ha az új gráfban el lehet érni a **maximális** D **folyamot** (azaz az s^* -ből induló összes él telítődik).



4. Hálózat alsó korláttal és követelménnyel

Ez a legösszetettebb eset. Az éleknek nem csak maximális, hanem **minimális** folyamértéke is van! Egy bizonyos mennyiséget *kötelező* átküldeni rajtuk.

Bemenet: $G(V, E, c, d, l)$, ahol

- $c : E \rightarrow \mathbb{N}$ kapacitás (felső korlát)
- $l : E \rightarrow \mathbb{N}$ **alsó korlát** (minimum ennyi árunak kell mennie)
- $d : V \rightarrow \mathbb{Z}$ követelmény (mint az előző feladatban)

Feltételek:

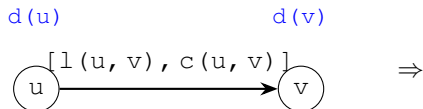
1. $l(e) \leq f(e) \leq c(e)$ (Minden él a megadott sávban operál).
2. $\sum_{(u,v) \in E} f(u,v) - \sum_{(v,w) \in E} f(v,w) = d(v)$

Kérdés: Van-e érvényes folyam?

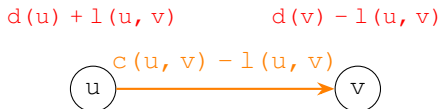
4. Alsó korlátok: az előre átküldés trükk

A cél, hogy megszabaduljunk az $[l(e), c(e)]$ korlátoktól. Ehhez rögtön az elején „erőnek erejével” átküldünk $l(e)$ mennyiséget minden élen, és csak a maradék szabad kapacitással dolgozunk tovább.

Eredeti él és igények



Módosított él és igények



Miért változnak a pont-igények?

- **u csúcs:** Mivel $l(u, v)$ egységet már „kiküldött”, ennyivel nőtt a tartozása a hálózat felé (vagy csökkent a kínálata).
- **v csúcs:** Mivel $l(u, v)$ egységet már „megkapott”, ennyivel kevesebb az eredeti igénye (vagy nőtt a kínálata).

4. Az alsó korlátos feladat visszavezetése

Ha minden élre elvégezzük a transzformációt, egy standard $G'(V, E, c', d')$ hálózatot kapunk alsó korlátok nélkül.

A globális módosítások összegezve:

- **Új élkapacitások:** $c'(e) = c(e) - l(e)$ minden éle.
- **Új csúcs-követelmények:** Minden v pontra az összes rajta átmenő alsó korlát módosítja az egyenleget:

$$d'(v) = d(v) + \sum_{(v,w) \in E} l(v,w) - \sum_{(u,v) \in E} l(u,v)$$

Megoldás lépései:

1. Az így kapott G' hálózatra alkalmazzuk a 3. pontban látott **superforrás-szupernyelő** (s^*, t^*) módszert.
2. Ha a kapott maximális folyam értéke D (telítődnek az s^* és t^* élei), akkor van megoldás.
3. Az eredeti hálózat folyamértékei: $f(e) = f'(e) + l(e)$.

Alkalmazások

1. Ki lehet bajnok? (Baseball Elimination)

A probléma: Adott egy N csapatos bajnokság, ahol csak a győzelemért jár pont. Az lesz a bajnok, aki **szigorúan több** győzelmet szerez, mint bárki más.

Ismert adatok:

- p_i : az i -edik csapat jelenlegi pontszáma (győzelmeinek száma).
- M : a még hátralévő mérkőzések száma.
- Egy adott hátralévő mérkőzés résztvevői: a_i és b_i csapatok.

A feladat: Döntsük el az N -edik (a saját) csapatunkról, hogy megnyerheti-e még a bajnokságot!

Megoldás: A legoptimistább forgatókönyv

Hogyan nyerhet a mi csapatunk (N)? Úgy, ha innentől kezdve mindent megnyer, a többiek pedig "számunkra kedvezően" játszanak.

Lépések:

1. **Számoljuk össze a mi meccseinket!** Tegyük fel, hogy az M hátralévő meccsből r darab olyan van, amiben az N -edik csapat **nem** játszik. Tehát nekünk $M - r$ meccsünk maradt.
2. **Maximális pontszámunk:** Ha az összeset megnyerjük, a végső pontszámunk:

$$P_{\max} = p_N + (M - r)$$

3. **A többiek korlátja ("keret"):** Ahhoz, hogy mi legyünk a bajnokok, senki sem érheti el a $P_{\max}$ pontot. Bármely $i < N$ csapat legfeljebb $P_{\max} - 1$ pontot szerezhet.
4. **Mennyit nyerhetnek még?** Az i -edik csapat a hátralévő meccseiből legfeljebb ennyit nyerhet meg:

$$d_i = P_{\max} - p_i - 1 = p_N + M - r - p_i - 1$$

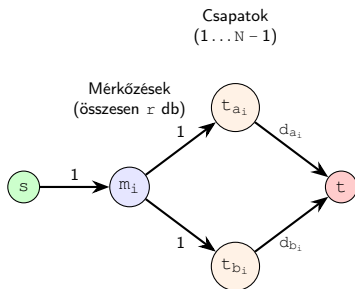
Fontos: Ha bármelyik i -re $d_i < 0$ adódik, azonnal kiesünk (már most több pontjuk van, mint amennyit mi maximálisan elérhetünk). Ha minden $d_i \geq 0$, jöhet a hálózati folyamat!

Megoldás: Hálózatépítés

A hátralévő r darab (nélkülünk játszott) meccs kimenetelét kell úgy "elosztanunk", hogy senki se nyerjen többet a megengedett d_i korlátjánál. Ezt egy $G = (V, E, c)$ gráffal modellezzük.

A gráf felépítése:

- **Pontok:** Szuperforrás (s), szupernyelő (t), minden meccsnek egy m_i pont, minden csapatnak egy t_i pont.
- **Élek és kapacitások:**
 - $s \rightarrow m_i$: kapacitás 1 (minden meccs 1 győzelmet ér).
 - $m_i \rightarrow t_{a_i}$ és $m_i \rightarrow t_{b_i}$: kapacitás 1 (a győzelmet a_i vagy b_i kapja).
 - $t_i \rightarrow t$: kapacitás d_i (a csapat $\max d_i$ további győzelmet szerezhet).



Tétel: Az N . csapat akkor és csak akkor lehet bajnok, ha ebben a gráfban a maximális folyam értéke pontosan r . (Azaz minden hátralévő mérkőzést le tudunk játszani úgy, hogy a kapacitás-korlátok ne sérüljenek)

2. Maximális lezárt

Definíció: A $G = (V, E)$ irányított gráf egy $C \subseteq V$ ponthalmaza **lezárt**, ha nincs belőle kivezető él:

$$(\forall u \in C) (\forall (u, v) \in E) \implies v \in C$$

A feladat:

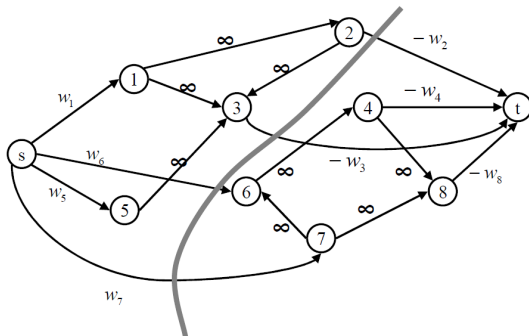
- Adott egy súlyozott gráf: $G = (V, E, \text{suly})$, ahol $\text{suly}(v) \in \mathbb{Z}$.
- Keressük azt a C lezárt halmazt, amelynek **maximális az összsúlya**.
- (Feltehető, hogy a gráf körmentes, különben az erősen összefüggő komponenseket összevonhatjuk).

Visszavezetés: Alkossunk egy $\bar{G} = (\bar{V}, \bar{E}, c)$ hálózatot:

- $\bar{V} = V \cup \{s, t\}$
- Ha $\text{suly}(v) > 0$: Vegyünk fel (s, v) élet, $c(s, v) = \text{suly}(v)$ kapacitással.
- Ha $\text{suly}(v) < 0$: Vegyünk fel (v, t) élet, $c(v, t) = -\text{suly}(v)$ kapacitással.
- Az eredeti élek: Minden $(u, v) \in E$ él marad, de $c(u, v) = \infty$ kapacitással.

Maximális lezárt – Szemléltetés

Visszavezetés minimális vágásra: nincs olyan éle az eredeti gráfnak, amely keresztezné a vágást.



Állítás: Legyen (S, T) a minimális vágás. Ekkor $C = S \setminus \{s\}$ a maximális súlyú lezárt az eredeti gráfban.

3. Projekt kiválasztás

A szituáció:

- n elvégezhető munka, minden munkának van egy w_i nyeresége (ami lehet negatív is, ha veszteséges).
- Vannak függőségek: $a_i \rightarrow b_i$ azt jelenti, b_i -t csak akkor végezhetjük el, ha a_i -t már befejeztük.
- Ha elvállalunk egy munkát, minden előfeltételét is (közvetett módon is) el kell végeznünk.

Modellezés:

1. Tekintsük a munkákat a gráf csúcsainak.
2. A függőségeket irányított élekként ábrázoljuk.
3. Egy megengedett munka-együttes pontosan egy **lezárt halmaz** a gráfban.
4. A cél a legnagyobb össznyereség $\implies$ **Maximális lezárt feladat.**

4. Táblás játék

A feladat: Egy N mezőből és M nyílból álló irányított körmentes gráfban (DAG) kell két menetet (utat) választanunk.

- **Menet:** Egy kezdőpozícióból (0-befok) egy célpozícióba (0-kifok) tartó út.
- **Speciális tulajdonság:** Bármely menet **ugyanannyi** (O darab) mezőt érint.
- **Cél:** Maximalizálni a két menetben érintett **különböző** mezők számát.

4. Táblás játék

A feladat: Egy N mezőből és M nyílból álló irányított körmentes gráfban (DAG) kell két menetet (utat) választanunk.

- **Menet:** Egy kezdőpozícióból (0-befok) egy célpozícióba (0-kifok) tartó út.
- **Speciális tulajdonság:** Bármely menet **ugyanannyi** (O darab) mezőt érint.
- **Cél:** Maximalizálni a két menetben érintett **különböző** mezők számát.

Észrevétel: Legyen a két út P_1 és P_2 .

A pontszám: $|P_1 \cup P_2| = |P_1| + |P_2| - |P_1 \cap P_2|$

Mivel minden út hossza O , a pontszám $2O - |P_1 \cap P_2|$.

A feladat tehát: keressünk két utat, amelyek a **lehető legkevesebb közös pontot** tartalmazzák!

Szeeparáló pontok: Az x pont szeeparáló, ha minden kezdőpontból induló menet érinti, és ő az egyetlen ilyen távolságra lévő pont. Ezeket a pontokat „muszáj” érinteni minden menetben.

Táblás játék – Modellezés hálózati folyamattal

A feladatot **pontduplázással** és maximális folyam keresésével oldjuk meg.

A hálózat felépítése ($\bar{G}$):

- Minden x mezőhöz két pont: x_{in} (x) és x_{out} ($x + n$).
- Élek és kapacitások:
 - **Belső él:** $x_{in} \rightarrow x_{out}$. Kapacitása 1, ha nem akarjuk többször számolni. Ha szeparáló, a kapacitás 2.
 - **Átmenő él:** Ha $(x, y) \in E$, akkor $x_{out} \rightarrow y_{in}$, kapacitás 1.
 - **Forrás/Nyelő:** $s \rightarrow s_{in}$ minden kezdőpontra, $e_{out} \rightarrow t$ minden célpontra.



Kapacitás szabály:

Ha x szeparáló: $c(x, x + n) = 2$

Egyébként: $c(x, x + n) = 1$

Megoldás menete: Futtassuk le a Ford-Fulkerson algoritmust pontosan két növelőút kereséséig. A két növelőút adja meg a két optimális menetet.

A két út előállításához a reziduális gráfban kell figyelnünk az élek állapotát.

Adatszerkezet az élekhez:

```
1 struct El {  
2     int v, c, f; // célpont, kapacitás, aktuális folyam  
3     int visszaut; // a párja indexe a szomszédsági listában  
4     bool eredeti; // igaz, ha az eredeti gráf éle (nem visszairányú)  
5 };
```

Az utak kinyerése:

1. Keressünk két növelőutat a szuperforrástól a szupernyelőig.
2. Minden növelésnél frissítsük a kapacitásokat a reziduális gráfban.
3. A folyam lefutása után keressünk két diszjunkt utat olyan éleken, ahol a folyam értéke pozitív (vagy a maradék kapacitás nulla az eredeti éleken).
4. **Fontos:** A pontduplázás miatt az utunk $x_{in} \rightarrow x_{out} \rightarrow y_{in} \rightarrow y_{out} \dots$ formátumú lesz, amiből csak minden második pont az eredeti mező.

Komplexitás: Mivel csak 2 növelőutat keresünk, a futásidő $O(M)$, ami bőven belefér a 100 000-es korlátba.

5. Két karakter, két szín

A feladat: Adott egy 0-kból és 1-esekből álló string. Minden karaktert pirosra vagy kékre kell festenünk.

- Ha az i . karakter **piros**: kapunk r_i ért.
- Ha az i . karakter **kék**: kapunk b_i ért.
- **Büntetés (Inverzió)**: Minden olyan piros karakterpár után, ahol az 1-es hátrébb van, mint a 0-ás ($i < j$, $s_i = 1$, $s_j = 0$), fizetünk 1 ért.

Cél: Maximalizáljuk az ért. számát!

5. Két karakter, két szín

A feladat: Adott egy 0-kból és 1-esekből álló string. Minden karaktert pirosra vagy kékre kell festenünk.

- Ha az i . karakter **piros**: kapunk r_i ért.
- Ha az i . karakter **kék**: kapunk b_i ért.
- **Büntetés (Inverzió)**: Minden olyan piros karakterpár után, ahol az 1-es hátrébb van, mint a 0-ás ($i < j$, $s_i = 1$, $s_j = 0$), fizetünk 1 ért.

Cél: Maximalizáljuk az ért számát!

A trükk: Alakítsuk át „veszteség-minimalizálássá”. Összes megszerezhető érme: $\sum (r_i + b_i)$. Ebből vonjuk le a minimális veszteséget:

- Ha pirosat választunk, „elveszítjük” a kékért járó b_i -t.
- Ha kéket választunk, „elveszítjük” a pirosért járó r_i -t.
- Ha két piros inverziót alkot, elveszítünk további 1 egységet.

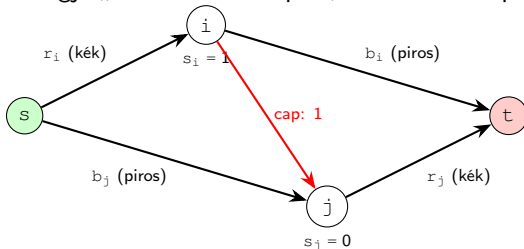
Modellezés minimális vágással

Egy olyan hálózatot alkotunk, amiben a vágás költsége pont a veszteségnek felel meg.

A minimális $s - t$ vágás során a pontokat S (forrás) vagy T (nyelő) oldalra osztjuk. Feleltessük meg ezeket a színeknek!

- Ha $s_i = 1$ (**1-es típus**): Piros $\in S$, Kék $\in T$.
- Ha $s_j = 0$ (**0-ás típus**): Piros $\in T$, Kék $\in S$.

Miért jó ez? Egy inverzió akkor történik, ha $s_i = 1$ piros ÉS $s_j = 0$ piros ($i < j$). A választott kódolással ez pontosan akkor teljesül, ha $i \in S$ és $j \in T$. Ha behúzzunk egy $i \rightarrow j$ élet 1 kapacitással, a vágás pontosan akkor fogja „büntetni” ezt a párt, ha mindkettő piros!



Bár a modell elméletileg tökéletes, a gráf éleinek száma $O(N^2)$ is lehetne az inverziók miatt, ami túl sok az eredeti feladat (Codeforces 1895G) korlátai mellett ($N = 4 \cdot 10^5$).

Hatékony megvalósítás:

- A hálózat speciális szerkezete lehetővé teszi a **mohó (greedy) megoldást**.
- Balról jobbra haladva próbálunk minél több folyamot átnyomni.
- Az 1-eseknél keletkező „fölösleges” kapacitást egy adatstruktúrában tároljuk, amiből a 0-ások „fogyasztanak”.
- **Adatszerkezet:** A feladat hatékonyan megoldható treap vagy szegmensfa segítségével.
- **Komplexitás:** $O(N \log N)$, ami a klasszikus Max-Flow-nál jóval gyorsabb ebben a speciális esetben.

Eredmény: Maximális profit = $\sum (r_i + b_i) - \text{Max Flow}$.

Feladatok

<https://open.kattis.com/problems/transportation>

Acmania országa több államból áll. Egyes államokban nyersanyag-beszállítók, másokban gyárak találhatóak. Szállítócégek vállalhatják az anyagok továbbítását bizonyos államok között.

Szabályok:

- Egy beszállító legfeljebb egy gyárat láthat el.
- Egy gyár legfeljebb egy beszállítótól kaphat nyersanyagot.
- Egy szállítócég legfeljebb egy útvonalban szerepelhet.
- A szállítás több cégen keresztül is történhet, ha két cég közös államban működik.

Feladat: Határozzuk meg, maximum hány gyár látható el nyersanyaggal.

Korlátok:

- $1 \leq r, f \leq 200$ (beszállítók és gyárak száma)
- $1 \leq s \leq 600$ (államok száma)
- $1 \leq t \leq 1000$ (szállítócégek száma)

Kattis – Avoiding the Apocalypse

<https://open.kattis.com/problems/avoidingtheapocalypse>

Egy városban egy fertőzés (zombi apokalipszis) elől kell minél több embert eljuttatni a kórházakba adott időn belül. A város helyszínei között utak vannak, amelyek különböző irányokban és különböző áteresztőképességgel működnek.

Szabályok:

- Az emberek a kezdő helyszínről indulnak.
- Minden út csak adott irányban járható.
- Egy út egyszerre csak korlátozott számú ember átengedésére képes (kapacitás).
- Az utak bejárása több időegységbe telik (utazási idő).
- Mindenki legfeljebb T időegységen belül kell, hogy elérjen egy kórházat.
- A csúcsokon korlátlan ideig lehet várakozni.

Feladat: Határozzuk meg, hogy a kezdő helyről induló csoportból hány ember tud legfeljebb T időn belül eljutni valamelyik kórházba.

Korlátok:

- Több tesztet, legfeljebb 100
- $1 \leq N, R \leq 1000$ (helyszínek és utak száma)
- $0 \leq T \leq 100$

Codeforces 1250K – Projectors

<https://codeforces.com/contest/1250/problem/K>

Az egyetemen n előadást és m szemináriumot tartanak különböző időintervallumokban. Van x HD projektor és y normál projektor.

Szabályok:

- Az előadásokon csak HD projektor, a szemináriumokon normál és HD projektor is használható.
- Egy projektor egyszerre legfeljebb egy eseményen használható.
- Ha egy esemény pontosan akkor kezdődik, amikor egy másik véget ér, ugyanaz a projektor újra felhasználható.
- Egy esemény teljes időtartamára ugyanazt a projektort kell használni.

Feladat: Döntsük el, hogy kioszthatók-e a projektorok az összes eseményhez a szabályok betartásával. Pozitív válasz esetén adjunk meg egy konkrét projektor-hozzárendelést is.

Korlátok:

- $0 \leq n, m, x, y \leq 300$
- $1 \leq a_i < b_i \leq 10^6$ (az előadások intervallumai)
- $1 \leq p_j < q_j \leq 10^6$ (a szemináriumok intervallumai)

https://atcoder.jp/contests/abc193/tasks/abc193_f

Adott egy $N \times N$ rács, ahol minden cella színe:

- **B**: fekete
- **W**: fehér
- **?**: még nincs kiszínezve (B vagy W választható)

Zebranness: Az egymás melletti (oldalszomszédos) cellapárok száma, amelyek különböző színűek (B–W).

Feladat: A ? cellák tetszőleges színezésével maximalizáljuk a zebranness értéket.

Korlátok:

- $1 \leq N \leq 100$
- $c_{i,j} \in \{B, W, ?\}$

Példa bemenet:

```
3
BBB
BBB
W?W
```

Példa kimenet:

```
4
```